マルチメディアプログラミング

担当教員：澤田

学籍番号：6324029
学年：3 年
氏名：北澤泰樹

提出日：2026 年 7 月 7 日

1　課題

1.1　課題１動画の読み込み

動画（People - 84973.mp4）を読み込み, 動画の縦・横のサイズ,FPS, 総フレーム数を出力してください.

1.2　課題２サムネイル画像の作成

サムネイルとして動画（People - 84973.mp4）中のフレーム画像を 1 枚出力し, 保存してください.
※画像は提出する必要はないですが、画像をレポートに含めること。
ただし、サムネイルは単に 1 フレーム目ではなく、以下のいずれかを選ぶ。
- 中央のフレーム- 自分で良いと思った選択方法で選んだ代表的なフレーム

1.3　課題３ ファイル名のパース

動画のファイル名（People - 84973.mp4）の文字列を処理して、タイトル（People）と ID（84973）に分けるプログラムを作成し、その結果を出力してください。

1.4　課題４ 色反転と動画の保存

動画（Cat - 66004.mp4）を各フレームの色を反転した動画を作成するプログラムを作成し、その動画を保存してください。

1.5　課題５ 音声の可視化

音声には時間-周波数領域にメッセージが隠されています.
パラメータを適宜調節して解析し，可視化したグラフを描画してください.

1.6　課題６ n-gram

この課題ではテキスト に出現する 2-gram の出現頻度を調べ、出現頻度が上位 10 個の 2-gram を対象に、横軸に 2-gram の語句を表示し、縦軸に出現頻度をとったグラフを描画してください。stop words 除去

1.7　課題 7 動画の変わり目を検出する

横軸が時間（フレーム），縦軸が変化量のグラフを描画してください．変化量がある値（閾値）よりも 大きいフレームを動画の変わり目としたとき，何番目のフレームが動画の変わり目かを出力してください.

1.8　課題 8 オブジェクト指向:video クラスの作成

Video クラスは 動画の path を受け取り、インスタンス化する。
Video クラスだけではなく、インスタンス化して動作を確認するプログラムもあわせて作成すること。

2　アルゴリズムの説明

2.1　動画の読み込み

指定されたパスから動画ファイルを読み込む。正常にファイルが開けたかを確認し、欠損等で開けなかった場合はエラーを返す。正常な場合は、動画の幅、高さ、フレームレート（FPS）、総フレーム数のメタデータを取得し、整数値に整形して出力する。最後にメモリリークを防ぐため、動画の読み込み状態を解放する。

2.2　サムネイル画像の作成

動画全体を代表する 1 フレームを抽出し、静止画として保存する。動画の冒頭はフェードインなどの演出が含まれることが多く、内容を正しく反映していない可能性が高いため、時間的に中央にあたるフレームを採用する。総フレーム数を 2 で割った位置のインデックスを算出し、その位置までシークしてフレームを取得・保存する。

2.3　ファイル名のパース

「タイトル - ID.mp4」という動画のファイル名の命名規則を利用し、メタデータを抽出する。まず拡張子を除去し、残りの文字列を「 - 」（スペース、ハイフン、スペース）をデリミタとして分割する。分割後の 0 番目の要素をタイトル、1 番目の要素を ID として分離し、それぞれ保持する。

2.4　動画の色反転と縮小保存

動画の全フレームに対して色反転とリサイズを行い、別ファイルとして保存する。出力用の解像度は元の幅と高さに 0.3 を乗じて算出する。各フレームの画素処理において、専用関数を用いず、画素値の最大値（255）から元の画素値を減算することで論理的に色反転（ネガポジ変換）を行う。その後、算出した解像度にリサイズし、出力ストリームへ順次書き込む。

2.5　音声の可視化

音声信号を短時間フーリエ変換（STFT）を用いて解析し、時間・周波数・信号強度の 3 次元情報をスペクトログラムとして可視化する。音声信号を短い窓関数で区切り、少しずつオーバーラップさせながら周波数成分を算出する。これにより、特定の周波数帯域における信号の強弱の時間変化を捉える。

2.6　n-gram 分析およびストップワード除去

テキストデータから連続する 2 単語の組（2-gram）を抽出し、出現頻度を分析する。まず、大文字・小文字の重複を防ぐためテキストを小文字に正規化し、記号類を除去して単語配列に分割する。課題 6-a では、そのまま隣り合う 2 単語を連結して 2-gram を生成する。課題 6-b では、事前に定義した代名詞や前置詞などのストップワードを除外した配列を生成してから 2-gram を抽出する。最後に双方の上位 10 件を抽出し、比較用のグラフを描画する。

2.7　動画の変わり目検出

隣接するフレーム間の画素値の変化量を利用して、シーンチェンジを自動検出する。計算コストと色相の影響を下げるため、各フレームをグレースケールに変換する。連続する t 番目と $t-1$ 番目のフレーム間で画素ごとの輝度差分の絶対値を算出し、その総和を各フレームの「変化量」とする。この変化量が、あらかじめ設定した閾値（固定値、または平均と標準偏差を用いた統計値）を超えた箇所をシーンの変わり目として特定する。

2.8　オブジェクト指向による動画管理クラスの作成

これまで個別に実装したファイル名のパース、サムネイル生成、動画加工などの機能を Video クラスとしてカプセル化する。コンストラクタでファイルパスを受け取り、パース結果などのメタデータをインスタンス変数として安全に保持する。また、動画の加工処理には高階関数のアプローチを採用する。フレームの読み込みやファイル書き出しといった共通のループ制御基盤に対し、個別の画素処理（色反転など）を関数オブジェクトとして外部から渡すことで、制御ロジックを書き換えることなく柔軟にフィルタを適用できるようにする。

3 プログラムの説明

3.1 動画の読み込み

Python の画像処理ライブラリである OpenCV（cv2）を用いて実装した。まず、cv2.VideoCapture() 関数の引数に動画のファイルパスを渡し、動画の読み込み用オブジェクト（cap）を生成する。続いて cap.isOpened() を用いて、動画ファイルが正常に開けたかを if 文で判定する。正常に開けなかった場合はエラーメッセージを出力する。

正常に読み込めた場合は、cap.get() メソッドを用いて各プロパティを取得する。引数に cv2.CAP_PROP_FRAME_WIDTH や cv2.CAP_PROP_FRAME_HEIGHT などの定数を指定することで、幅、高さ、FPS、総フレーム数をそれぞれ取得する。幅、高さ、総フレーム数は小数点以下を持たないデータとして扱うため、int() を用いて整数型に変換して変数に格納した。

最後に各結果を標準出力し、メモリリークを防ぐために必ず cap.release() を呼び出してシステムリソースを解放している。

3.2 サムネイル画像の作成

OpenCV の cap.get() メソッドに対し、定数 cv2.CAP_PROP_FRAME_COUNT を指定することで動画の総フレーム数を取得した。中央のフレームを特定するため、取得した値を 2 で整数除算（// 演算子）し、その結果を変数 middle_frame_idx に格納した。続いて、cap.set() メソッドを用いて読み込み位置の制御を行った。第 1 引数に cv2.CAP_PROP_POS_FRAMES、第 2 引数に算出したインデックスを指定することで、指定したフレーム番号まで読み込み位置をスキップさせている。移動後の位置において cap.read() を実行して 1 フレーム分の画像データ（frame）を取得し、cv2.imwrite() 関数を用いて Google ドライブ上の指定パスへ保存した。正常に動作したことを視覚的に確認するため、Google Colab 環境専用の cv2_imshow() を用いて実行結果画面に抽出画像を表示する処理を実装した。

3.3 ファイル名のパース

Python の標準ライブラリである os モジュール、および文字列操作メソッドを用いて実装した。まず、対象となるファイル名が格納された文字列変数 file_name に対し、os.path.splitext() 関数を適用した。これにより、拡張子より前の文字列（name_without_ext）と拡張子（ext）に分離させた。次に、得られた name_without_ext に対して .split(" - ") メソッドを実行した。この処理により、「-」を境界として文字列が分割され、リスト型変数 parts に格納される。リストのインデックス 0 番目の要素（parts[0]）を変数 title に、1 番目の要素（parts[1]）を変数 video_id にそれぞれ代入することで、データのパース（解析）を実現した。最後に、f-string（フォーマット文字列）を用いて、指定された形式（Title: および ID:）に整形して標準出力を行った。

3.4 動画の色反転と縮小保存

OpenCV のライブラリを用いて、動画の再エンコード処理を実装した。まず cap.get() メソッドにより取得した CAP_PROP_FRAME_WIDTH および HEIGHT に対して 0.3 を乗じ、int() でキャストして新しい解像度

を決定した。動画の書き出しには cv2.VideoWriter() クラスを使用した。引数には、指定された形式に準拠した出力パス（6324029_cat_reverse.mp4）、cv2.VideoWriter_fourcc(*'mp4v') で指定したコーデック、元動画から継承した FPS、および算出済みの新解像度を渡した。while ループ内では、ブロードキャスト機能を利用して 255 - frame という行列演算を行うことで色反転を実現した。続けて cv2.resize() 関数を用いて線形補間による縮小を実行した。加工されたフレームは out.write() メソッドによって順次ファイルへ記録される。最後に cap.release() および out.release() を実行し、ファイルハンドルを安全に閉じる構成とした。

3.5 音声の可視化

Python の科学技術計算ライブラリである scipy.io.wavfile とグラフ描画ライブラリ matplotlib を用いて実装した。まず wavfile.read() 関数を用いて音声ファイルを読み込み、サンプリング周波数（fs）と音声データ（data）を取得した。グラフの描画には plt.specgram() 関数を使用し、Fs 引数に取得したサンプリング周波数を指定した。音声に隠されたメッセージをより鮮明に可視化するため、FFT の窓サイズを決定する NFFT を 2048 に、窓の重なり幅を決定する noverlap を 1024 に設定した。また、視認性を高めるため、カラーマップに 'inferno' を採用し、plt.colorbar() で色の強度を示すスケールバーを追加した。これにより、音声データ内に隠されていた時間-周波数領域のメッセージを視覚的に特定できた。

3.6 n-gram 分析およびストップワード除去

Python の標準コレクションモジュールである Counter および matplotlib ライブラリを統合して実装した。指定のパスからファイルをオープンして読み込んだ後、.lower() メソッドで小文字化を行い、.replace() メソッドをループさせることで記号類を取り除いた。その後、.split() によってスペース区切りの単語リスト words を作成した。6-a では、リスト内包表記を用いて words[i] と words[i+1] を結合した文字列のリスト bigrams_raw を作成した。6-b では、事前に定義した集合型 stop_words に対し、内包表記の条件式 if w not in stop_words を用いて不要語を排除した filtered_words を生成した。これに対して同様の隣接連結を行い、bigrams_filtered を構築した。それぞれの出現頻度の集計には Counter() クラスを適用し、.most_common(10) メソッドを呼び出すことで、出現頻度上位 10 組の 2-gram とその出現回数を辞書型として抽出した。グラフ化の際は、plt.subplots(2, 1) により上下 2 段のキャンバスを確保し、それぞれの keys() を横軸、values() を縦軸とした棒グラフを ax.bar() で描画した。x 軸ラベルの視認性を確保するため、tick_params() で文字を 15 度回転させる処理を加えた。

3.7 動画の変わり目検出

Python の OpenCV および Numpy の行列演算機能を用いて、関数制限を満たすロジックを実装した。while ループ内で cv2.cvtColor() を用いて各フレームをグレースケール化した後、前後の差分計算を行った。符号なし 8 ビット整数型（uint8）のまま減算を行うと負の値がアンダーフローを発生させるため、.astype(np.int16) を用いて一時的に符号付き 16 ビット整数型にキャストした。そのうえで、np.abs() による絶対値演算を行い、さらに np.sum() を適用することで、全画素の差分の総和（変化量）を一挙に算出して配列 diff_values に格納した。7-a のグラフ描画では、plt.plot() を用いてフレームごとの変化量の推移を可視化した。7-b の閾値設計では、1 つ目の手法として変化量の最大値に 0.6 を乗じた値を固定の

境界とする threshold_fixed を定義した。2 つ目の手法として、全体の平均値（np.mean()）に標準偏差（np.std()）の 5 倍を足し合わせた値を統計的境界とする threshold_stat を定義した。それぞれの閾値条件を満たすインデックスを np.where() によって抽出し、フレーム番号に補正したうえで標準出力させた。また、検出されたフレーム位置まで cap.set() でシークし、cv2.imwrite() を用いて確認用画像を指示されたディレクトリ（dataset/images/output/）に保存する機構を実装した。

3.8 オブジェクト指向による動画管理クラスの作成

Python のクラス構文およびオブジェクト指向プログラミング手法を用いて Video クラスを実装した。コンストラクタ __init__() 内で self.path、self.title、self.video_id、self.thumbnail の 4 つのインスタンス変数を宣言した。parse_filename() では os.path.basename() 経由で取得したファイル名を前述の課題 3 のロジックでパースし、結果を直接 self.title と self.video_id に代入した。get_info() メソッドでは、これら 4 つの変数をキーとした辞書（dict）型オブジェクトを構築して return するよう実装した。高階関数となる process_video() メソッドは、第 2 引数としてフレーム処理用の関数ポインタ（filter_func）を受け取る設計とした。ループ内（while True）で cap.read() により取得した生フレームに対し、引数経由の filter_func(frame) を動的に実行する構成をとった。この関数ポインタの実体として、クラス内に @staticmethod デコレータを用いた reverse_filter()、gray_filter()、binary_filter() の 3 種の画像処理関数を定義した。メインの動作確認ブロックでは、Video のインスタンスを生成後、各メソッドを順次呼び出し、最後に video_instance.process_video(output_path, Video.reverse_filter) として関数を引数に渡す形で、指定された出力ディレクトリ（dataset/videos/output/）への動画再生成が正常に行われることを検証した。

4　実行結果

4.1　動画の読み込み

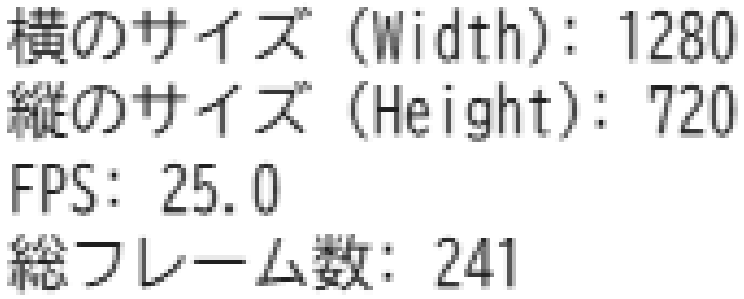


図 1: 実行結果

4.2 サムネイル画像の作成

```
...  サムネイルを保存しました: /content/drive/MyDrive/Colab Notebook
抽出したフレーム番号: 120
```

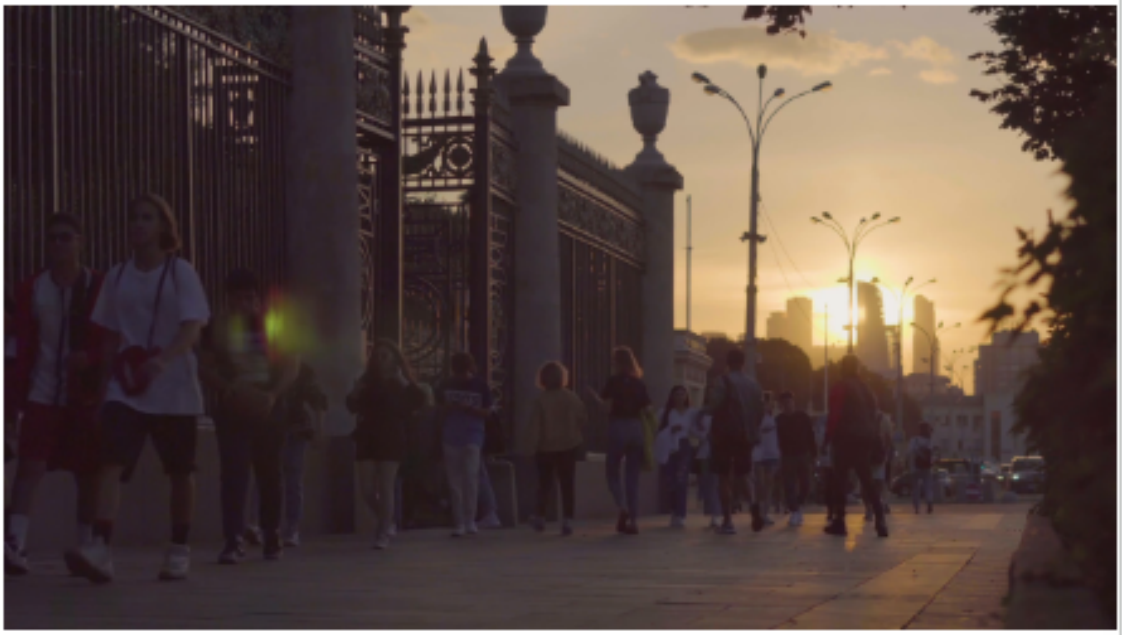


図 2: 実行結果

4.3 ファイル名のパース



図 3: 実行結果

4.4　音声の可視化

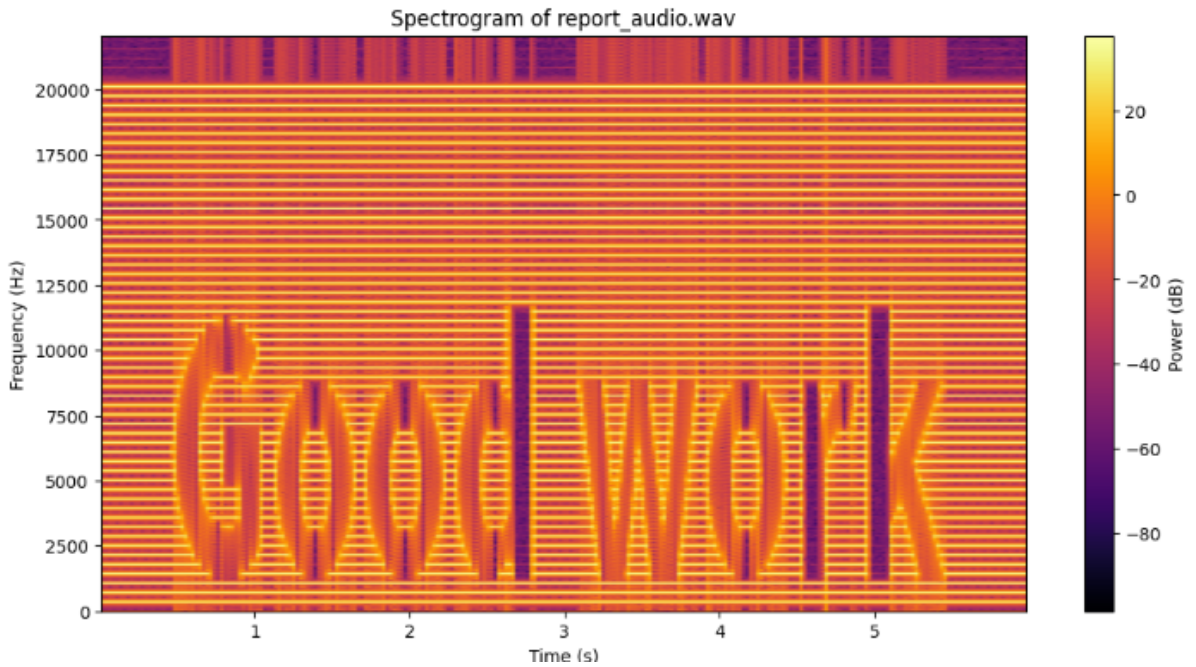


図 4: 音声ファイル（report_audio.wav）のスペクトログラム。隠されたメッセージが読み取れる。

4.5 n-gram

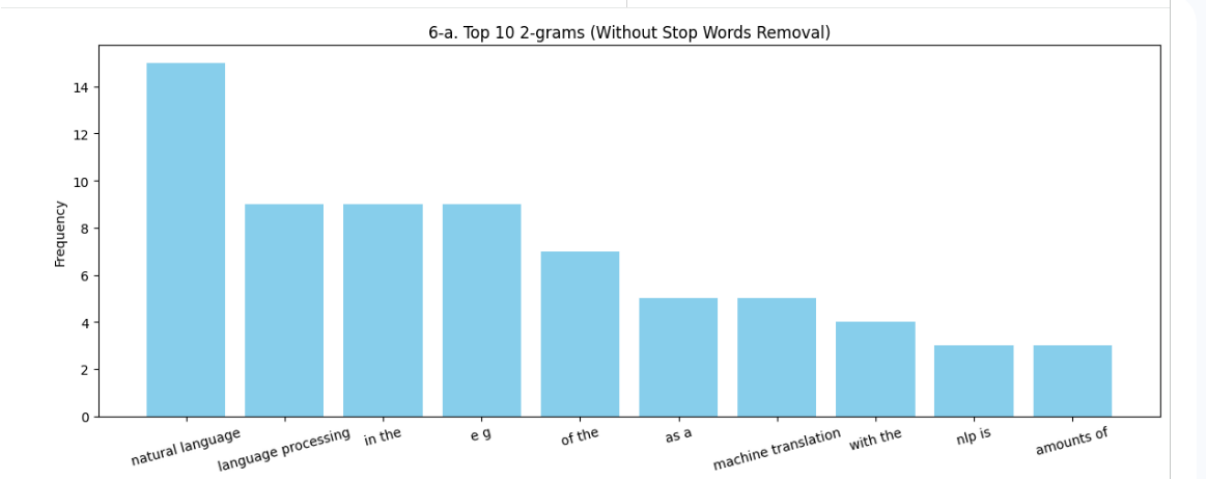


図 5: 棒グラフ（除去なし）

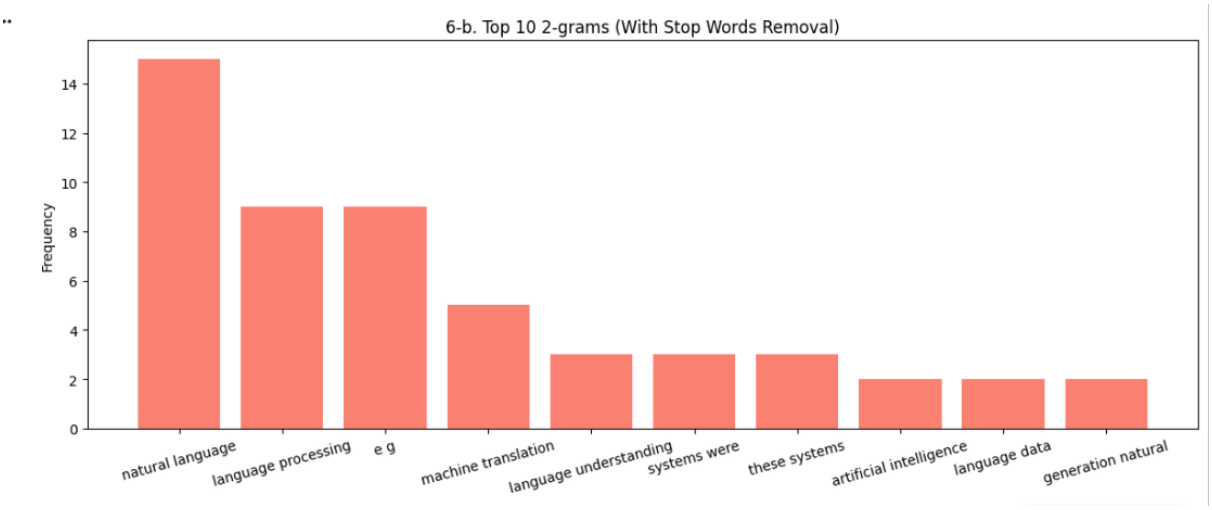


図 6: 棒グラフ（除去あり）

4.6 動画の変わり目検出

Listing 1: 動画の変わり目検出のテキスト出力結果

```
==========================================
 7-b. 動画の変わり目検出結果
==========================================【方法
1: 固定値（閾値: )】175391647.8閾値を超えたフレーム番号
:
 [78, 214, 257]【方法

2: 平均 + 標準偏差（閾値5*: )】115336923.8閾値を超えたフレーム番号
:
 [78, 214, 257]【本当に変わり目か確認用の画像保存】フレーム


 78 の画像を保存しました -> /content/drive/MyDrive/Colab Notebooks/dataset/images/
    output/scene_change_78.jpgフ
    レーム
 214 の画像を保存しました -> /content/drive/MyDrive/Colab Notebooks/dataset/images/
    output/scene_change_214.jpgフ
    レーム
 257 の画像を保存しました -> /content/drive/MyDrive/Colab Notebooks/dataset/images/
    output/scene_change_257.jpg
```

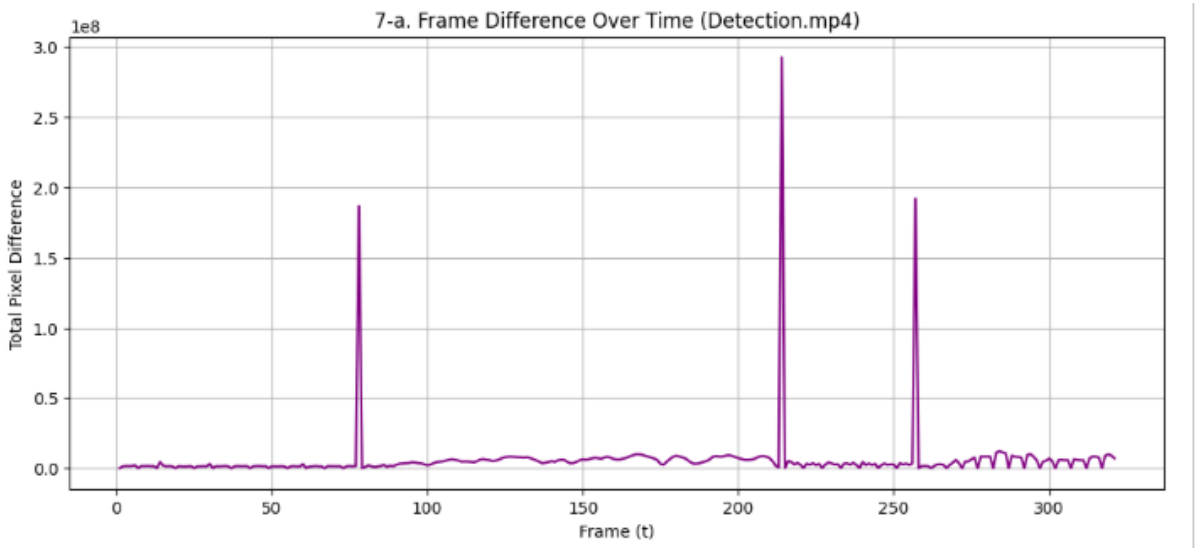


図 7: フレーム間の画素値変化量の時間推移グラフ

4.7　オブジェクト指向による動画管理クラスの作成

```
--- get_info() の出力結果 ---
Path:      /content/drive/MyDrive/Colab Notebooks/dataset/videos/Cat - 66004.mp4
Title:     Cat
ID:        66004
Thumbnail (Shape): (720, 1280, 3)
------------------------------高階関数による動画処理を実行します

...動画処理が完了しました。保存先
: /content/drive/MyDrive/Colab Notebooks/dataset/videos/output/6324029
    _cat_processed.mp4
```

5　考察

5.1　サムネイル画像抽出におけるフレーム選択の妥当性（課題２）

課題２において、動画の代表画像として中央フレームを選択した。動画の冒頭は製作上の演出として暗点やフェードイン、アウトなど、またタイトルロゴなどが挿入されることが多い。そのため第一フレームを機械的に抽出しても動画の代表として不適切な可能性がある。一方で動画の中央付近は被写体や背景のシーンが安定している可能性が高い。したがって、1 枚の静止画像で動画全体の内容を要約するというサムネイルの目的において、中央うれーむの自動選択は、合理的なアプローチであると言える。

5.2　スペクトログラムにおける解像度と視認性の相関（課題５）

音声ファイルに隠されたメッセージを解読するにあたり、STFT のパラメータの窓サイズ NFFT と重なり幅 noverlap が視認性に与える影響を比較した。最も鮮明にメッセージが読み取れたのは NFFT=2048, noverlap=1024 の設定である。窓サイズを大きく取ることで周波数分解能が向上し、メッセージを構成する文字の輪郭、特に縦方向の境界線がノイズから分離されたためと考えられる。逆に NFFT=256 のように窓サイズを小さくすると、時間分解能は向上するが周波数分解能が低下し、データが周波数方向にぼやけて判別不能となった。隠蔽メッセージの解読においては、時間領域と周波数領域のトレードオフを考慮し、周波数分解能を優先したパラメータ設計が有効であることが確認できた。

5.3　n-gram 分析におけるストップワード除去の効果（課題 6）

テキストデータの特徴抽出において、前処理（ストップワード除去）がもたらす効果を検証した。除去を行わない場合、出現頻度上位は「the of」や「in the」といった、構文上頻出する冠詞や前置詞の組み合わせに占有された。これらは文法的に必須であっても、テキストの主題を表す情報量はほぼ皆無である。一方、これらの不要語を除外する処理を施すことで、「natural language」や「language processing」といった、対象テキストの内容を直接的に示す名詞句が上位に浮上した。この結果から、自然言語処理の実務において、データの量的なノイズを減らし質的な特徴を抽出するためには、適切なフィルタリング工程が不可欠であることがわかった。

5.4　動画の変わり目検出における閾値設計の根拠（課題７）

シーンの境界を自動検出するため、特性の異なる 2 種類の閾値判定を設計し有効性を比較した。1 つ目の固定閾値（最大変化量の 0.6 倍）は、動画内でシーンチェンジが最も劇的な画素変化を伴うという前提に基づいている。最大ピークに対する相対値を基準とすることで、被写体の移動などによる局所的なノイズを抑え、主ピークのみを高速に捉えることができた。2 つ目の統計的閾値（平均 + 5σ）は、定常的な変化を正規分布に従うノイズとみなし、そこから逸脱した異常値をシーンチェンジとするアプローチである。標準偏差の 5 倍という基準により、通常のフレーム間差分を超える突発的な変化のみを動的に検出できた。結果として双方で同一のフレーム番号を検出できたが、特に後者の統計的アプローチは、事前に変化量の最大値が不明な未知の動画に対しても適応できるため、より堅牢なシステム設計の根拠になると言える。

5.5　オブジェクト指向による動画管理の有用性と機能比較（課題８）

マルチメディア処理を独立した関数の集まりで記述する場合と、オブジェクト指向としてカプセル化する場合の設計上の違いについて考察する。第一に、データ管理の安全性が挙げられる。手続き型では動画のパスやタイトル等が散在しやすく、意図しない変数の上書きを誘発しやすい。クラス化することで、各動画の属性がインスタンス変数として安全に保持され、アクセサメソッドを介すことで状態管理が容易になる。第二に、高階関数の導入による再利用性の向上である。手続き型で新たな動画加工処理を追加する場合、動画の I/O 制御コードごと複製する必要があり冗長になる。本実装のように、動画制御基盤のメソッドにフィルタ関数（画素処理ロジック）を引数として注入する設計にすれば、制御コードを変更することなく、関数を 1 つ追加するだけで無限の画像エフェクトに拡張できる。これらの点から、オブジェクト指向を用いた「状態の隠蔽」と「関心の分離」は、システムの保守性や拡張性を高める上で非常に有用である。